LC 74 search in a 2d matrix

Approach:

- Convert the 2d matrix in to a 1d single sorted array where we can apply binary search to search and find the target element
- Map t 1d index to corresponding row and column to 2d matrix where
    - row=mid/sizeof(matrix column)-which row the element belongs to.
    - col = mid % sizeof(matrix column) → which column inside that row.

LC 50 Pow(x,n)

Approach:

- This is a classic problem focused on recursion and maths
- If n is 0, answer is always 1
- If n is 1, the output is the number itself
- If n is even recursive divide the power with 2 and multiply the number with itself for n/2 times
- If n id odd, reduce the power by one to make it even and use that 1 as a base case and multiply it with the resulted even power answer
- To handle negative powers, take reciprocal of positive result

LC 169 Majority element

Approach:

- We initialize two variables
    - Count -> to count the number of times an element appears
    - Element -> to store the array element
- Store first element when count=0 then make count=1
- If current element of array is same as element , increment count by 1
- If current element of array is not element, decrement count by 1
- Return element
    - Or
- We can also use hashmap and compare key vale pairs and return element with maximum repetitions